Section 3: @Transactional and readOnly = true

Ridesharing Backend — Learning Notes

---

1. What is a database transaction?

A transaction is a group of database operations treated as a single unit — either ALL succeed or NONE do. No partial results.

Real ridesharing example — driver accepts ride:

- UPDATE ride_requests SET status = 'MATCHED', driver_id = 7
- UPDATE driver_profiles SET is_available = false WHERE id = 7
- INSERT INTO ride_offers_log (offer_id, action) VALUES (42, 'ACCEPTED')

All three must succeed together. If step 2 fails, step 1 must be rolled back — otherwise you have a MATCHED ride with a driver who still appears available.

ACID Properties

Property	Ridesharing meaning
Atomicity	All or nothing — all 3 updates succeed or all roll back
Consistency	DB moves from one valid state to another
Isolation	Two drivers can't both accept the same ride simultaneously
Durability	Committed data survives crashes and restarts

2. What @Transactional does in Spring

Without @Transactional, every SQL statement auto-commits immediately in its own mini-transaction. A failure halfway through leaves partial data permanently written.

With @Transactional, Spring wraps the entire method in one transaction using a proxy. It opens the transaction before your method runs and commits (or rolls back) when it finishes.

```
@Transactional  // opens transaction BEFORE method body runs
public void acceptRide(Long rideId, Long driverId) {
    RideRequest ride = rideRepo.findById(rideId).orElseThrow();
    ride.setStatus(RideStatus.MATCHED);
    ride.setDriverId(driverId);

    Driver driver = driverRepo.findById(driverId).orElseThrow();
    driver.setAvailable(false);

    // RuntimeException here → BOTH updates rolled back
}   // method returns normally → BOTH updates committed
```

3. Dirty checking in Hibernate

When you load an entity inside a transaction, Hibernate stores a snapshot of its original state in the persistence context (first-level cache). At commit time, Hibernate compares every loaded entity against its snapshot. If anything changed, it fires an automatic UPDATE — even if you never called save().

```
@Transactional
public void acceptRide(Long rideId, Long driverId) {
    RideRequest ride = rideRepo.findById(rideId).orElseThrow();
    // Hibernate snapshot: {status=OPEN, driverId=null}

    ride.setStatus(RideStatus.MATCHED);
    ride.setDriverId(driverId);
    // No save() called!

    // At commit: Hibernate compares to snapshot, detects change, fires:
    // UPDATE ride_requests SET status='MATCHED', driver_id=7 WHERE id=42
}
```

Key terms

- Persistence context = the tracking scope for one transaction.
- First-level cache = another name for the persistence context.
- Dirty = an entity whose current state differs from its snapshot.
- Dirty checking = the comparison Hibernate runs at commit time.

4. What readOnly = true does

When you mark a transaction readOnly = true, two concrete things happen:

1. Hibernate skips snapshot creation — no dirty check at commit time. For a method loading 50 entities, this means 50 fewer snapshot comparisons. Less memory, less CPU.
2. The database driver receives a read-only hint. MySQL can skip acquiring certain row locks and use read-optimised execution paths.

Common misconception

- readOnly = true does NOT skip the commit.
- The transaction still opens and closes normally.
- What changes: Hibernate never tracks snapshots, so there is nothing to flush at commit.
- The commit happens — it just has zero pending changes to write.

```
@Transactional(readOnly = true)  // no snapshot, no dirty check
public RideRequest getRideDetails(Long rideId) {
    return rideRepo.findById(rideId).orElseThrow();
}

@Transactional(readOnly = true)
public List<RideRequest> getPassengerHistory(Long passengerId) {
    return rideRepo.findByPassengerId(passengerId);
}
```

5. When to use which

Annotation	Use for
@Transactional	acceptRide(), createRideRequest(), cancelRide()
@Transactional(readOnly = true)	getRideDetails(), getPassengerHistory(), estimateFare()

Best practice — class-level default with method overrides:

```
@Service
@Transactional(readOnly = true)  // default for ALL methods
public class RideService {

    @Transactional  // overrides class default — writes to DB
    public void acceptRide(Long rideId, Long driverId) { ... }

    @Transactional  // overrides class default — writes to DB
    public RideRequest createRideRequest(CreateRideRequestDTO dto) { ... }

    // Inherits readOnly = true from class level
    public RideRequest getRideDetails(Long rideId) { ... }

    // Inherits readOnly = true from class level
    public List<RideRequest> getPassengerHistory(Long passengerId) { ... }
}
```

6. Rollback behaviour

Exception type	Spring behaviour
RuntimeException (unchecked)	Automatic rollback — default behaviour
Checked Exception	NO rollback by default — use rollbackFor
rollbackFor = Exception.class	Rollback on ALL exceptions including checked
noRollbackFor = X.class	Suppress rollback for a specific exception

```
// Checked exception example — payment fails, roll back ride status too
@Transactional(rollbackFor = Exception.class)
public void processPayment(Long rideId) throws PaymentException {
    RideRequest ride = rideRepo.findById(rideId).orElseThrow();
    ride.setStatus(RideStatus.COMPLETED);
    paymentService.charge(ride.getPassengerId(), ride.getFare());
    // If PaymentException thrown: ride status change is rolled back
}
```

Section 3 — what you learned

- Transactions group SQL operations — all succeed or all roll back (ACID).
- @Transactional wraps your method in one transaction via a Spring proxy.
- Without it, each SQL auto-commits — partial failures leave corrupted data.
- Dirty checking: Hibernate snapshots entities on load, auto-UPDATEs at commit.
- readOnly = true skips snapshot creation and dirty checking — faster reads.
- readOnly does NOT skip the commit — the transaction still closes normally.
- Class-level @Transactional(readOnly=true) + method-level @Transactional is best practice.
- Spring rolls back on RuntimeException only — use rollbackFor for checked exceptions.

Next: Section 4 — Optimistic Locking with @Version